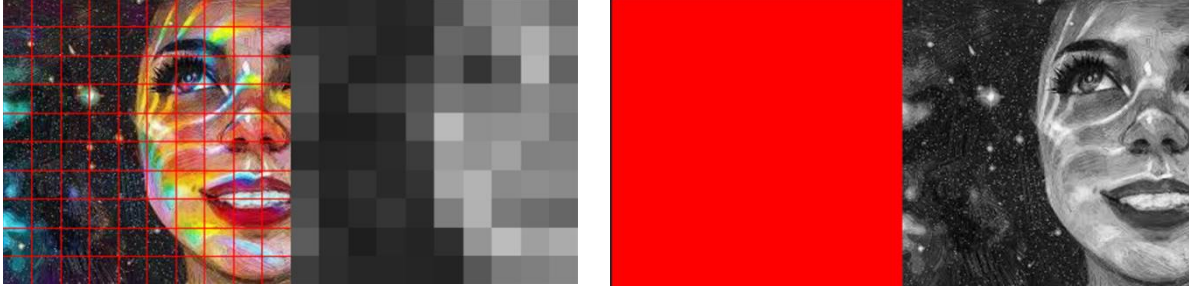


Bu proje, standart bir görsel dosyasını (JPEG/PNG) piksel tabanlı analiz ederek önce sayısal verilere, ardından metin tabanlı sembollere dönüştüren ve bu sayede yüksek oranda sıkıştırma sağlayan özgün bir yazılım sürecidir. Süreç temel olarak iki ana evreden oluşur: **Kodlama (Encoding/Sıkıştırma)** ve **Kod Çözme (Decoding/Geri Oluşturma)**.

1. Evre: Görüntünün İşlenmesi ve Veri İndirgeme

Süreç, ham görüntünün sisteme yüklenmesiyle başlar. Görüntü, kullanıcı tarafından belirlenen satır ve sütun sayılarına göre sanal bir ızgaraya (grid) bölünür. Bu aşamada amaç, görüntünün çözünürlüğünü optimize edilebilir bloklara ayırmaktır.



Görsellerden biri 10x10 olarak bölünmüş diğeri ise 255x255 Bölünmüştür

Her bir ızgara karesinin (bloğun) içerisindeki pikseller taranır ve bu piksellerin **ortalama gri tonlama değeri** (0-255 arası) hesaplanır. Böylece binlerce pikselden oluşan bir blok, tek bir sayısal değer ile temsil edilir hale gelir. Bu işlem sonucunda görsel, bir sayı matrisine dönüşür.

Örnek Çıktı:

175, 175, 175, 175, 175, 175, 175, 175, 176, 176, 176, 177 #Satır1

175, 175, 175, 175, 175, 175, 175, 175, 176, 176, 176, 177 #Satır2

175, 175, 175, 175, 175, 175, 175, 175, 176, 176, 176, 177 #Satır3

Ancak 0-255 aralığındaki sayılar (örneğin "245" veya "175") metin dosyasında 3 karakterlik yer kaplar. Veri boyutunu düşürmek için **Oranlı Küçültme** işlemi uygulanır. Renk uzayı 0-255'arasına indirgenerek 0-255 verisi 0-25 gibi gösterilir.

- Örnek: Orijinal değer **250** ise, bu **25** olarak kaydedilir.
- Örnek: Orijinal değer **34** ise, bu **3** olarak kaydedilir.

Böylece veri aralığı 0-255'ten, 0-25'e indirilmiş olur. (Not çözüm kısmında 10 ile çarpılarak eski değerini alır)

17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17 #Satır1

17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17 #Satır2

17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17 #Satır3

1.Evre Durum Özeti

Görüntü işleme adımımda resmi ızgaralara bölerek renk verilerini ayrıştırdık ve metin tabanlı olarak kaydettik. Bu süreçteki dosya boyutu değişimleri şu şekildedir:

- **Orijinal Dosya (jpg):** 13 KB
- **İlk Çıktı (işlediğimiz veri 0-255):** 210 KB
- **Optimize Edilmiş Çıktı (0-25):** 118 KB

Değer aralığını daraltarak dosya boyutunu yarı yarıya düşürmeyi başardık (210 KB -> 118 KB). Ancak orijinal görselin (JPG) 13 KB olduğu göz önüne alındığında, mevcut yöntem henüz yeterli verimliliğe ulaşmamıştır. 13 KB hedefine yaklaşmak için farklı sıkıştırma stratejileri üzerinde çalışılması gerekmeyi bize zorunlu kılmıştır.

2. Evre: Birinci Katman Sıkıştırma (Baktığım kadarıyla benzer mantıkta Modifiye RLE ile)

Elde edilen Sayı dizisi üzerinde, tekrar eden verileri sıkıştırmak için. Algoritma satırları tarar ve yan yana gelen aynı karakterleri sayar.

- **Durum:** Yan yana 5 tane '17' var;
- **Eski Yöntem:** 17 17 17 17 17 (10 karakter + boşluklar)
- **Yeni Yöntem:** 517 (3 karakter)

Bir önceki adımda elde ettiğim veri:

17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17 #Satır1

17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17 #Satır2

17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17 #Satır3

Yeni Kod çıktısı:

1217 #Satır1 yani 12 tane 17 var demek

1217 #Satır2

1217 #Satır3

Bu yöntemle, özellikle gökyüzü veya düz duvar gibi görselde az detay içeren, düz renkli alanlar inanılmaz derecede küçük boyutlara indirgenir.

2.Evre Durum Özeti

Uygulanan optimizasyon adımları sonucunda dosya boyutundaki değişim şu şekildedir:

- **Orijinal Dosya:** 13 KB
- **1. Aşama:** 210 KB
- **2. Aşama:** 118 KB
- **3. Deneme (Sonuç):** 99 KB

Değerlendirme: Dosya boyutu ilk kez 100 KB'ın altına düşürülmüştür. 210 KB'dan 99 KB'a inmek önemli bir başarı olsa da, 13 KB hedefine ulaşmak için yöntem değişikliğine ihtiyaç duyulmaktadır.

3. Evre: Verinin Sembolleştirilmesi (Mapping)

Sayısal veriler hala metin dosyasında yer kaplamaktadır (örneğin "17" iki karakterdir). Veriyi daha da sıkıştırmak için bu sayılar **Alfasayısal Karakterlere** dönüştürülür. 0 ile 25 arasındaki her sayıya İngilizce alfabesinden bir harf atanır. İngilizce seçmem zaten 25 e kadar yapmam ve harf 26 olması benim için tam uygun. 0 dan başlarsak tam benlik.

- 0 - A
- 1 - B
- ...
- 12 - M
- 25 - Z

Bu işlem sayesinde, çift haneli sayılar tek bir harf ile ifade edilerek dosya boyutunda anında %50'ye varan bir tasarruf sağlanır.

Eski çıktı:

Yeni çıktı:

1217 # Satır1

12R # Satır1

1217 # Satır2

12R # Satır2

1217 # Satır3

12R # Satır3

Şimdi ilk yaptığım veri işlemeden şuna kadar yaptığım sıkıştırmayı kıyaslayan görseli paylaşmak istiyorum



Sıkıştırma oranının kıyaslanması

Sıkıştırma Sürecinde Geline Son Nokta

Adım adım uygulanan iyileştirmeler sonucunda verilerdeki değişim tablosu şöyledir:

- **Orijinal Dosya:** 13 KB (Hedef)
- **1. Aşama:** 210 KB
- **2. Aşama:** 118 KB
- **3. Aşama:** 99 KB
- **4. Deneme (Güncel):** 46 KB

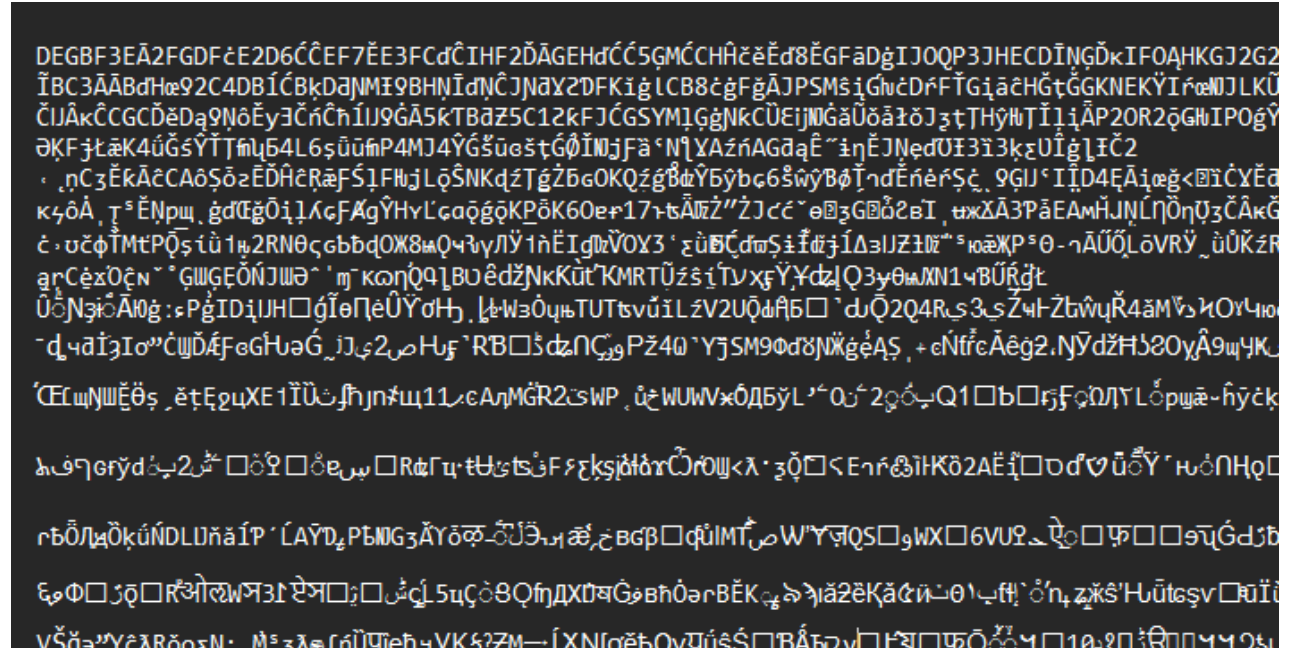
Değerlendirme: İlk denemeye göre dosya boyutunu 210 KB'dan 46 KB'a indirerek çok ciddi bir kazanım sağladık. Ancak orijinal görselin (13 KB) verimliliğini yakalamak için aradaki farkı kapatacak yeni yöntemlere ihtiyacımız var.

4. Evre: İkinci Katman Sıkıştırma (Sözlük Tabanlı/LZW)

Burada **LZW (Lempel-Ziv-Welch)** algoritması mantığına benzer bir yapı kullanılır. Algoritma, metin içindeki tekrar eden desenleri (örneğin 4M3Z deseni metinde 50 kere geçiyorsa) öğrenir. Bu uzun desenleri, ASCII tablosunda yer almayan ancak Unicode setinde bulunan tek bir karaktere (örneğin bir Çince veya Japonca sembole) dönüştürür.

- Girdi: 4M3Z (4 Karakter)
- Çıktı: hk (Tek bir Unicode karakteri)

Bu işlem sonucunda elde edilen veri.txt dosyası açıldığında, insan gözüyle anlamsız görünen sembollerden oluşur, ancak bilgisayar için en yoğun sıkıştırılmış hali ifade eder.



Çıktı örneği

Geliştirdiğimiz sıkıştırma algoritmasının evrimi ve dosya boyutundaki çarpıcı değişim:

- **Orijinal Dosya (Hedef):** 13 KB

- **1. Aşama:** 210 KB
- **2. Aşama:** 118 KB
- **3. Aşama:** 99 KB
- **4. Aşama:** 46 KB
- **SONUÇ:** ~16 KB □

Değerlendirme: Başlangıçtaki 210 KB'lık hantal veriyi, kalite kaybını minimize ederek 16 KB'a kadar sıkıştırdık. Orijinal .jpg formatıyla neredeyse aynı boyuta ulaşarak, metin tabanlı işlemede maksimum verimliliği yakaladık.

5. Evre: Geri Çözme ve Görüntü Oluşturma (Reconstruction)

Sıkıştırılmış dosyanın tekrar görsel dönüşürmesi işlemi tersine mühendislik ile yapılır:

Sonuç olarak elde edilen piksel değerleri, boş bir tuval üzerine kareler (bloklar) halinde çizilerek orijinal görüntü yeniden oluşturulur.

